Short-Term Action-observation Memory for FlashSAC in Dexterous Manipulation

Zheng-Lin Wu
Dept. of CSIE
National Taiwan University
Taipei, Taiwan
b11705045@ntu.edu.tw

Jian-Bin Wu
Dept. of CSIE
National Taiwan University
Taipei, Taiwan
b12902071@ntu.edu.tw

Chih-En Kuo
Dept. of CSIE
National Taiwan University
Taipei, Taiwan
b11902171@ntu.edu.tw

I-Chen Lee
Dept. of CSIE
National Taiwan University
Taipei, Taiwan
b11902168@ntu.edu.tw

Yu-Lin Chu
Data Science Degree Program
National Taiwan University
Taipei, Taiwan
r14946003@ntu.edu.tw

Abstract

Dexterous manipulation poses a challenging setting for deep reinforcement learning, as effective control requires reasoning over high-dimensional dynamics, intermittent contacts, and delayed consequences of previous actions. Most off-policy methods for continuous control, including FlashSAC, rely on feed-forward actor and critic networks that condition primarily on the current observation. In this work, we investigate whether explicitly incorporating short-term interaction history improves learning in dexterous cube reorientation. We introduce Short-Term Action-observation Memory (STAM), a lightweight recurrent context module that encodes recent observation-action trajectories via an LSTM and uses the resulting representation to modulate actor and critic features through a gating mechanism, leaving the underlying feed-forward FlashSAC backbone intact. We evaluate STAM-FlashSAC against feed-forward FlashSAC on the primary Shadow Hand OpenAI-FF cube reorientation environment in IsaacLab, and conduct ablations over action-history inclusion and context length. On this primary environment, STAM-FlashSAC converges faster during training and improves the final evaluation return over FlashSAC by roughly 35% within a 50M-step training budget, with all 10 paired seeds improving over FlashSAC. Additional ablations show that the improvement is not explained by simple history concatenation or parameter count alone. Our results provide empirical evidence that short-term action-observation history can improve feed-forward off-policy learning in the primary Shadow Hand setting, while broader generalization requires additional multi-task evaluation. Code is available here.

1 Introduction

Dexterous manipulation is a challenging problem for deep reinforcement learning because successful control requires high-dimensional coordination, stable contact interactions, and adaptation to delayed physical effects. In cube reorientation, a robotic hand must rotate and stabilize an object through many small finger motions. Even with state-based observations, the current state may not fully reveal the recent actions and contact transitions that produced it. This makes short-term history especially important in contact-rich manipulation.

National Taiwan University Fall 2026 ML-MiniConf (Oral presentation).

Off-policy actor–critic methods such as FlashSAC are attractive for this setting because they reuse past experience and support high-throughput training. However, the standard FlashSAC actor and critic are feed-forward networks that condition primarily on the current observation or state–action pair. This design is efficient, but it does not explicitly use recent interaction history. Previous actions may provide useful conditioning about recent finger motions, object momentum, and contact changes.

In this work, we study whether adding short-term action-observation memory improves FlashSAC for dexterous cube reorientation. We introduce Short-Term Action-observation Memory (STAM), a lightweight recurrent context module that encodes a fixed-length window of recent observations and previous actions with an LSTM. The resulting context representation is used to gate hidden features in the actor and critic, leaving the main FlashSAC backbone feed-forward. This design allows FlashSAC to incorporate local temporal information without becoming a fully recurrent off-policy algorithm.

We evaluate STAM-FlashSAC primarily on the IsaacLab Shadow Hand OpenAI-FF cube reorientation task, comparing it with a feed-forward FlashSAC baseline. We also perform ablations over context length and action-history inclusion to determine whether previous actions provide useful information beyond observation-only memory. We additionally include related-environment checks in the appendix to clarify boundary conditions for STAM beyond the primary Shadow Hand OpenAI-FF setting.

Our results show that action-observation memory improves FlashSAC more reliably than observation-only memory in the primary Shadow Hand OpenAI-FF setting. Ablations show that both context length and previous-action history affect performance, supporting the role of short-term action-observation memory in the primary setting. In the main 10-seed paired comparison, STAM-FlashSAC with $K = 16$ and action history improves the final evaluation return over feed-forward FlashSAC by 35.54% on the primary Shadow Hand OpenAI-FF cube reorientation environment. Additional ablations compare STAM against frame stacking, encoder variants, and gating versus concatenation, supporting the role of learned short-term memory rather than simple input expansion alone.

The main contributions of this work are as follows:

- We propose STAM, a lightweight recurrent context module that adds short-term action-observation memory to FlashSAC while preserving its feed-forward actor and critic backbone.
- We show that including previous actions in the STAM context improves over observation-only memory in the primary Shadow Hand setting.
- We demonstrate that STAM-FlashSAC improves final evaluation return over feed-forward FlashSAC by 35.54% in a 10-seed paired comparison on the primary Shadow Hand OpenAI-FF cube reorientation environment.
- We analyze the effects of context length, action-history inclusion, and frame stacking in the primary setting, while reporting additional architecture and cost analyses in the appendix.

2 Related Work

2.1 Off-Policy Reinforcement Learning for Continuous Control

Deep reinforcement learning is widely used for continuous robotic control, where agents must learn policies over high-dimensional state and action spaces. Policy-gradient methods such as PPO [1] are robust and easy to parallelize, while off-policy methods such as SAC and FlashSAC are attractive for high-throughput robotic control because they reuse replay data. Our main baseline is FlashSAC, a recent off-policy actor–critic method designed for high-throughput robotic control [2]. FlashSAC builds on SAC [3] while using larger models, fewer gradient updates, and norm-control mechanisms to improve stability in high-dimensional manipulation. In this work, STAM-FlashSAC preserves the feed-forward FlashSAC backbone and adds STAM as a lightweight short-term memory module.

2.2 Dexterous Manipulation in Simulation

Dexterous manipulation benchmarks in GPU-based simulators such as Isaac Gym and IsaacLab [4, 5] enable large-scale training and evaluation for contact-rich robotic control. We focus on the Isaac-Repose-Cube-Shadow-OpenAI-FF-Direct-v0 task from IsaacLab. This state-observation setting is

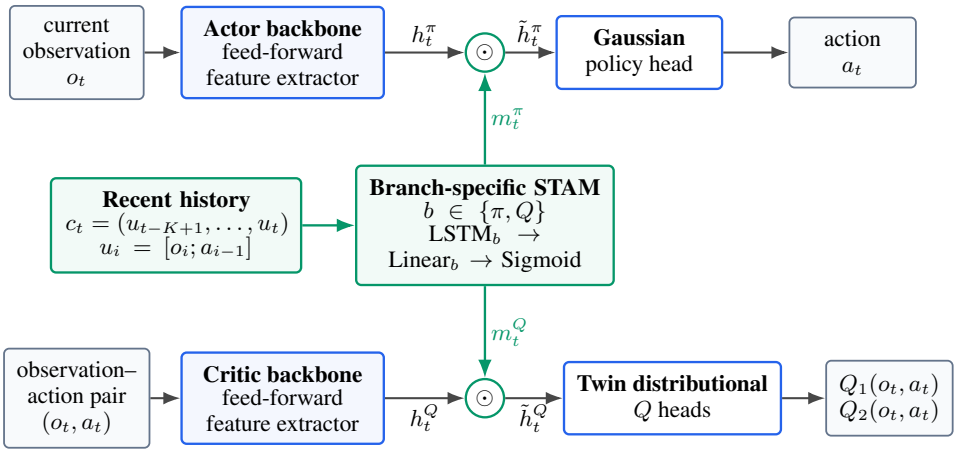


Figure 1: Overview of **STAM-FlashSAC**.

useful for isolating the effect of short-term action-observation memory, since it avoids additional confounds from visual encoders or image representation learning.

2.3 Temporal Context Modeling

Many control tasks are not fully determined by a single observation: recent actions can reveal momentum, delayed control effects, and contact transitions. Recurrent policies can model such temporal information, but fully recurrent off-policy training often increases implementation and optimization complexity. RISE shows that recent history can instead be encoded separately and used to condition a primarily feed-forward controller [6]. STAM follows this general direction but targets a different regime: state-based actor–critic control with FlashSAC rather than image-based value learning. It also stores matched current and next context windows in replay for bootstrap targets and evaluates action-observation memory as a lightweight modulation branch within FlashSAC.

3 Methodology

This work builds on FlashSAC, a high-throughput off-policy actor–critic method, and augments it with **Short-Term Action-observation Memory (STAM)**, a lightweight recurrent module that conditions the policy and value function on a short window of recent observations and previous actions. The resulting method is referred to as **STAM-FlashSAC**. We first review the components of FlashSAC that are relevant to our study (Section 3.1), then describe the proposed STAM module (Section 3.2), and finally specify the training objective used in our experiments (Section 3.3).

3.1 FlashSAC Background

FlashSAC is an off-policy maximum-entropy actor–critic method designed to remain stable under large-batch, high-throughput training with massively parallel simulators. We retain the FlashSAC backbone unchanged and attach STAM as an external memory-modulation branch; the components relevant to our study are summarized below.

Spherical networks and distributional critic. Every linear map in the actor and critic is bias-free, orthogonally initialized, and re-projected onto the unit sphere after each gradient step ($w \leftarrow w/\|w\|_2$), with normalization-layer parameters rescaled analogously to norm $\sqrt{d}$. This weight- and feature-norm control keeps the network scale bounded and supports stable large-batch optimization. Each network consists of a stem, a stack of inverted-residual blocks (a $4\times$ expansion with BatchNorm, ReLU, and a residual connection), an RMSNorm, and an output head. The critic is distributional, following the categorical distributional RL formulation [7]: it predicts a categorical return distribution over $N = 101$ atoms on $[V_{\min}, V_{\max}] = [-5, 5]$, trained with a distributional Bellman backup (Section 3.3), and rewards are scaled online to keep returns within the atom support across tasks.

Ensemble and policy. The critic maintains two value heads evaluated in parallel through a batched linear operator, and clipped double-Q learning [8] is applied at the distribution level by selecting the smaller-valued member to form the target. The actor is a feed-forward $\tanh$-squashed Gaussian policy whose entropy temperature α is tuned automatically toward a fixed, embodiment-independent target entropy $\bar{\mathcal{H}} = \frac{1}{2}|\mathcal{A}| \log\big(2\pi e\, \sigma_{\text{tgt}}^2\big)$ with $\sigma_{\text{tgt}} = 0.15$. We otherwise retain the original FlashSAC exploration and entropy-temperature tuning mechanisms.

3.2 STAM: Short-Term Action-observation Memory

The FlashSAC actor and critic described above are feed-forward and condition primarily on the current observation or state–action pair. To inject short-term temporal information, we introduce *Short-Term Action-observation Memory (STAM)*, a lightweight recurrent module for state-based dexterous control. STAM encodes recent action-observation history and converts the resulting context representation into feature-wise modulation signals. The guiding principle is that temporal memory should *modulate* the feed-forward features rather than replace them: the FlashSAC backbone remains the primary feature extractor, and STAM only reweights its hidden activations. Figure 1 illustrates how the memory branch attaches to the feed-forward actor and critic.

Context construction. At each time step, STAM constructs a context window c_t from the most recent K observation-action tokens. Each token is defined by concatenating the current observation with the previous action,

$$u_t = [\, o_t \,;\, a_{t-1} \,] \in \mathbb{R}^{d_o + d_a}, \tag{1}$$

and the full context is a fixed-length window of the most recent K tokens, $c_t = (u_{t-K+1}, \ldots, u_t)$, updated by a first-in-first-out shift at every step. The window is zero-initialized at the start of training and is reset for any environment whose episode terminates, so that a new episode does not inherit stale history. We treat the inclusion of the previous action as a design choice: in the observation-only variant the token reduces to $u_t = o_t$, and we ablate this choice in Section 4.

STAM modulation. The context window c_t is encoded by a single-layer LSTM. For each branch $b \in \{\pi, Q\}$, the final LSTM state is mapped by a linear layer and sigmoid nonlinearity to a modulation vector $m_t^b \in (0,1)^h$ with the same width as the corresponding backbone feature,

$$z_t^b = \mathrm{LSTM}_b(c_t), \qquad m_t^b = \sigma\big(W_m^b z_t^b + b_m^b\big)\,. \tag{2}$$

Let h_t^π and h_t^Q denote the actor and critic hidden features immediately before their output heads, where h_t^π is produced from o_t and h_t^Q from (o_t, a_t). STAM applies the modulation elementwise,

$$\tilde{h}_t^\pi = h_t^\pi \odot m_t^\pi, \qquad \tilde{h}_t^Q = h_t^Q \odot m_t^Q. \tag{3}$$

The modulated features are then passed to the RMSNorm and their respective output heads. Since $m_t^b \in (0,1)^h$, STAM acts as a conservative feature-wise gate: it attenuates or preserves existing backbone features rather than replacing the feed-forward representation.

Placement in actor and critic. We attach STAM to both the actor and the critic. In the actor, STAM modulation is inserted between the residual backbone and the Gaussian action head. In the critic, STAM acts on the ensemble features: the projection of the LSTM hidden state produces one modulation vector per ensemble member, which modulates the corresponding member's features before the categorical value head. The actor and critic STAM modules use the same context construction but do not share recurrent or projection parameters with each other or with the FlashSAC backbone. The recurrent module is small relative to the backbone, and because it is evaluated once per forward pass over a short K-step window, it adds only a modest overhead to the high-throughput training loop, which we quantify empirically in Section 4.

Context in the replay buffer. Because the method is off-policy, the context window must be consistent between data collection and training. We therefore store, for each transition, both the context window c_t observed at the time of acting and the next-step context window c_{t+1} formed by appending the realized (o_{t+1}, a_t) token. At update time, c_t is used for the current state and c_{t+1} for the bootstrap target, mirroring exactly how the windows are produced during rollouts. This avoids recomputing histories from sampled minibatches and keeps the recurrent inputs identical between acting and learning.

3.3 Training

Objective. Training follows the standard distributional maximum-entropy actor–critic recipe. For STAM-FlashSAC, the actor and critic additionally condition on the context window c_t through the STAM modulation branch. The critic is fit with a cross-entropy loss against a projected n-step distributional Bellman target,

$$\mathcal{T}z = \sum_{k=0}^{n-1} \gamma^k \bar{r}_{t+k} + \gamma^n (1 - d_{t+n}) \big(z - \alpha \log \pi(a_{t+n} \mid o_{t+n}, c_{t+n})\big), \tag{4}$$

using online-scaled rewards $\bar{r}$, clipped double-Q at the distribution level, and a Polyak-averaged target critic. The actor, updated every two critic steps, maximizes the clipped-minimum expected value minus the entropy term, and α is tuned toward the embodiment-independent target $\bar{\mathcal{H}}$. Spherical weights and normalization parameters are re-projected after every update. The one detail specific to our pipeline is that, to keep BatchNorm statistics consistent between the prediction and the bootstrap target, the current and next state–action pairs are passed through the critic in a single concatenated forward call.

Training regime. Unless otherwise stated, we follow the FlashSAC high-throughput configuration: 1024 parallel environments, a replay buffer holding up to 10^6 transitions, a batch size of 2048, an update-to-data ratio of two gradient updates per environment interaction step, a discount $\gamma = 0.99$, a soft-update coefficient $\tau = 0.01$, and an Adam learning rate of 3×10^{-4} annealed by cosine decay. Mixed-precision and graph compilation are enabled for throughput. We keep the FlashSAC actor and critic architecture unchanged and add STAM as an external memory-modulation branch. This lets us study short-term action-observation memory without replacing the feed-forward FlashSAC backbone.

4 Experiments

We design our experiments to answer three questions. (i) Does adding short-term observation–action memory to FlashSAC improve learning on the primary Shadow Hand OpenAI-FF cube reorientation task? (ii) How do the memory design choices—context length K and the inclusion of previous actions—affect performance? (iii) How does STAM compare with simple frame-stacking baselines? Throughout, we report results with multiple seeds and emphasize interval estimates rather than single-run point values.

4.1 Experimental Setup

Environment. `Isaac-Repose-Cube-Shadow-OpenAI-FF-Direct-v0` cube reorientation task from IsaacLab [5], in which a Shadow Hand must rotate a cube to a sequence of target orientations, is used in our main experiments. This is a high-dimensional, contact-rich control problem and is one of the state-based dexterous manipulation tasks used to evaluate FlashSAC.

Methods compared. We compare the feed-forward *FlashSAC baseline* [2], i.e. the backbone of Section 3.1 with the memory branch disabled, against our *STAM-FlashSAC* variants. The main STAM ablations vary the context length $K \in \{4, 8, 16, 32\}$ and whether the context token contains observations and previous actions or observations only. All FlashSAC variants share the high-throughput training configuration detailed in Section 3.3 and summarised in Appendix C.

Protocol. Each configuration is trained for 50M environment steps. For the main FlashSAC versus STAM-FlashSAC comparison, we use 10 paired random seeds. Ablations use 3 seeds unless otherwise stated. We evaluate periodically with a deterministic policy and report the mean evaluation return averaged over $1{,}024$ episodes. Learning curves show the mean across seeds with shaded 95% confidence intervals; bar charts report the final evaluation return with error bars over seeds. Additional implementation details, hyperparameters, and appendix-only control experiments are reported in the appendix.

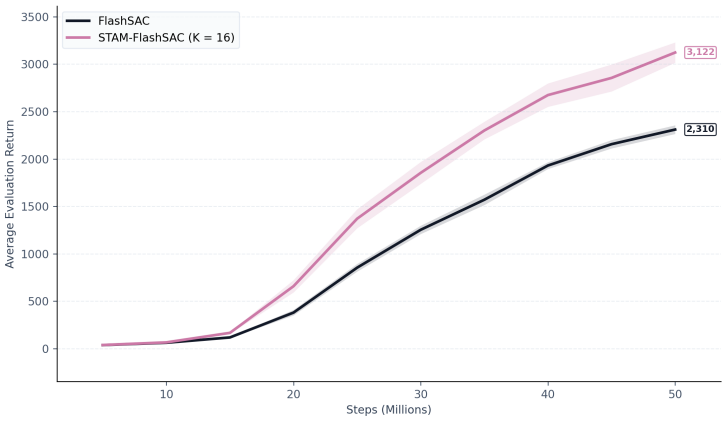


Figure 2: Learning curves on Shadow Hand OpenAI-FF task across 10 paired seeds. Shaded regions show 95% confidence intervals.

4.2 STAM Improves FlashSAC on Shadow Hand

Figure 2 compares the FlashSAC baseline and our default STAM configuration ($K = 16$) on Shadow Hand cube reorientation across 10 paired seeds. Against this strong off-policy baseline, STAM-FlashSAC learns faster throughout training and reaches a substantially higher final return on the primary Shadow Hand OpenAI-FF environment, improving the mean final evaluation return from 2,310 to 3,122. This corresponds to a mean paired improvement of 812.43 final-return points and a relative gain of 35.54%. Appendix D reports the full seed-level statistics for this paired comparison.

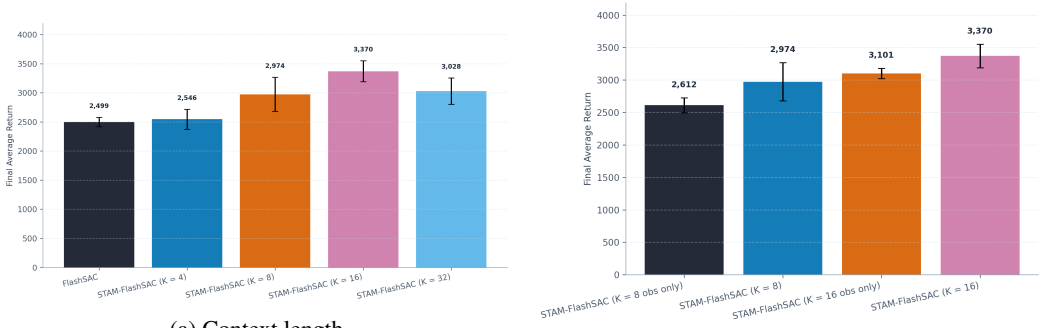
4.3 Ablation Study: Context Length and Action History

To understand which components of STAM contribute to the final performance gain, we conduct three ablation studies on the Shadow Hand cube reorientation task. We first vary the context length K, then isolate the effect of including previous actions in the context token, and finally examine sensitivity to the LSTM hidden dimension. Additional architecture-placement and temporal-encoder ablations are reported in Appendix I and Appendix H.

Effect of context length. Figure 3(a) compares the feed-forward FlashSAC baseline with STAM-FlashSAC using context lengths $K \in \{4, 8, 16, 32\}$. Compared with the FlashSAC baseline return of 2,499, STAM-FlashSAC achieves 2,546 with $K = 4$, 2,974 with $K = 8$, 3,370 with $K = 16$, and 3,028 with $K = 32$, corresponding to relative improvements of approximately 1.9%, 19.0%, 34.9%, and 21.2%, respectively. The small gain at $K = 4$ suggests that very short histories contain limited information about recent contact dynamics. Longer windows provide more useful temporal context, but extending the context further to $K = 32$ does not improve over the $K = 16$ setting in our experiments. Since $K = 32$ also increases replay-context storage and training cost, we use $K = 16$ as the practical default for the remaining experiments.

Effect of action history. Figure 3(b) further compares observation-only memory against action-observation memory for $K = 8$ and $K = 16$. Including previous actions consistently improves performance at both context lengths. For $K = 8$, adding action history increases the final average return from 2,612 to 2,974, a relative improvement of approximately 13.9%. For $K = 16$, it increases the return from 3,101 to 3,370, corresponding to an additional 8.7% improvement. These gains show that previous-action tokens are useful in STAM. Appendix F reports a shuffled-action control that further probes this effect.

Effect of LSTM hidden dimension. We ablated hidden dimensions $\{64, 128, 256\}$ with $K = 16$ fixed. Hidden $= 128$ achieves the best 3-seed mean final return (2,999.90); hidden $= 64$ is close (2,966.38, $\Delta = -33$) and hidden $= 256$ is lower (2,802.03, $\Delta = -198$). In this 3-seed ablation, increasing the hidden dimension to 256 does not improve over 128; we therefore keep 128 as a



(a) Context length

(b) Action history

Figure 3: Context-length and action-history ablations on Shadow Hand OpenAI-FF.

reasonable default. The appendix provides the corresponding hidden-dimension comparison in Appendix G.

Temporal encoder choice. We use a single-layer LSTM as the default STAM encoder. Appendix H compares LSTM, GRU, and flattened-history MLP encoders under the same $K = 16$ action-observation memory and feature-gating setup. The results suggest that STAM's benefit is not tied to a specific recurrent cell.

4.4 Comparison with Frame Stacking Baselines

To distinguish learned recurrent memory from simple history access, we run a separate 5-seed comparison between STAM-FlashSAC and two $K = 16$ frame-stacking baselines on the Shadow Hand OpenAI-FF environment. The first baseline concatenates the past $K = 16$ observations, while the second concatenates both past observations and previous actions. These baselines expose the policy to recent history without introducing a recurrent memory module.

Both frame-stacking variants improve over the FlashSAC baseline, indicating that temporal context is important for this task. However, STAM achieves the strongest overall performance. As summarized in Table 1, STAM reaches the highest final return, improving over FlashSAC by 45.0%, while the two frame-stacking baselines improve final return by 22.5% and 37.3%, respectively. STAM also obtains the highest AUC return, suggesting better overall learning efficiency across training rather than only better final performance. Appendix E includes the corresponding learning curves and additional cost details.

Table 1: Performance on the Shadow Hand OpenAI-FF environment across 5 seeds.

Method	Final Return (mean ± std)	Final Gain	AUC (mean ± std)
FlashSAC	2,351.83 ± 224.69	–	1,040.86 ± 82.97
FrameStack ($K = 16$, obs + action)	2,880.98 ± 413.37	+22.5%	1,095.45 ± 215.00
FrameStack ($K = 16$, obs only)	3,229.24 ± 375.80	+37.3%	1,323.31 ± 81.75
STAM-FlashSAC ($K = 16$)	**3,409.15 ± 237.63**	**+45.0%**	**1,602.86 ± 134.73**

Interestingly, observation-only frame stacking outperforms observation-action frame stacking, suggesting that naive action concatenation may introduce harder-to-optimize inputs. In contrast, STAM learns a recurrent memory representation and achieves the best final return and AUC.

5 Conclusion

We studied whether Short-Term Action-observation Memory (STAM) can improve FlashSAC on dexterous cube reorientation. STAM adds a lightweight recurrent memory branch that encodes recent observations and previous actions and uses the resulting signal to modulate actor and critic features, while preserving the feed-forward FlashSAC backbone.

On the primary Shadow Hand OpenAI-FF environment, STAM-FlashSAC improves the mean final evaluation return from $2{,}310$ to $3{,}122$, a relative gain of 35.54% in the 10-seed paired comparison. The improvement is consistent across paired seeds, and our ablations show that both context length and action history matter. Under the default $K = 16$ setting, including previous actions improves over observation-only memory. These results support the value of action-observation memory over observation-only memory in the primary setting.

Appendix results on Shadow Direct and Allegro Direct suggest that STAM's benefit is environment-dependent, motivating broader multi-task evaluation. Overall, STAM provides a simple way to add short-term action-observation memory to FlashSAC without replacing its feed-forward backbone. On the primary Shadow Hand OpenAI-FF task, this memory branch improves final evaluation return over a strong FlashSAC baseline, while broader generalization across tasks and embodiments remains an important direction for future work.

6 Member Workload

All team members contributed to the project development, experimental evaluation, paper writing, and reviewer response.

- **Zheng-Lin Wu**: Contributed to experiments, manuscript writing, and reviewer response. Additionally led the initial idea formulation, coordinated the overall project direction and experimental plan, organized the rebuttal and final revision process, and prepared the project presentation.
- **Jian-Bin Wu**: Contributed to experiments, result analysis, manuscript writing, and reviewer response.
- **Chih-En Kuo**: Contributed to experiments, result analysis, manuscript writing, and reviewer response.
- **I-Chen Lee**: Contributed to experiments, result analysis, manuscript writing, and reviewer response.
- **Yu-Lin Chu**: Contributed to experiments, result analysis, manuscript writing, and reviewer response.

References

[1] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.

[2] Donghu Kim, Youngdo Lee, Minho Park, Kinam Kim, I Made Aswin Nahendra, Takuma Seno, Sehee Min, Daniel Palenicek, Florian Vogt, Danica Kragic, Jan Peters, Jaegul Choo, and Hojoon Lee. Flashsac: Fast and stable off-policy reinforcement learning for high-dimensional robot control. *arXiv preprint arXiv:2604.04539*, 2026.

[3] Tuomas Haarnoja, Aurick Zhou, Pieter Abbeel, and Sergey Levine. Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor. In *Proceedings of the International Conference on Machine Learning*, 2018.

[4] Viktor Makoviychuk, Lukasz Wawrzyniak, Yunrong Guo, Michelle Lu, Kier Storey, Miles Macklin, David Hoeller, Nikita Rudin, Arthur Allshire, Ankur Handa, and Gavriel State. Isaac gym: High performance gpu-based physics simulation for robot learning. In *Advances in Neural Information Processing Systems Datasets and Benchmarks Track*, 2021.

[5] Mayank Mittal, Pascal Roth, James Tigue, Antoine Richard, Octi Zhang, Peter Du, Antonio Serrano-Muñoz, Xinjie Yao, René Zurbrügg, Nikita Rudin, Lukasz Wawrzyniak, et al. Isaac lab: A gpu-accelerated simulation framework for multi-modal robot learning. *arXiv preprint arXiv:2511.04831*, 2025.

[6] Tyler Clark, Christine Evers, and Jonathon Hare. Recurrent off-policy deep reinforcement learning doesn't have to be slow. *arXiv preprint arXiv:2512.20513*, 2025.

[7] Marc G. Bellemare, Will Dabney, and Rémi Munos. A distributional perspective on reinforcement learning. In *Proceedings of the International Conference on Machine Learning*, 2017.

[8] Scott Fujimoto, Herke van Hoof, and David Meger. Addressing function approximation error in actor-critic methods. In *Proceedings of the International Conference on Machine Learning*, 2018.

A Related-Environment Results on the Shadow Hand Direct Variant

To examine whether the effect of STAM remains visible beyond the main evaluation environment, we additionally test STAM-FlashSAC on the `Isaac-Repose-Cube-Shadow-Direct-v0` environment for 200M environment steps. Compared with the main environment, the Direct variant uses a different and richer observation formulation while retaining the same Shadow Hand cube reorientation objective. It therefore provides a related setting for evaluating whether the benefit of short-term memory remains observable when the baseline policy already receives more informative state observations.

Figure 4 shows the learning curves of the FlashSAC baseline and STAM-FlashSAC on the Shadow Hand Direct variant. Both methods improve steadily over training and eventually reach strong final performance. After extending the training horizon to 200M steps, STAM-FlashSAC achieves a slightly higher final return than the FlashSAC baseline, improving the mean final return from 12,099.38 to 12,600.16, corresponding to a 4.1% gain. More importantly, STAM-FlashSAC also improves training efficiency, as reflected by the area under the learning curve, increasing the mean AUC return from 9,624.94 to 9,887.14, a 2.7% improvement. Table 2 summarizes these results. These findings suggest that even in the Direct variant, where the observation space is already relatively informative, incorporating short-term observation-action history can still provide useful sample-efficiency benefits.

Table 2: Performance on the Shadow Hand Direct environment over 200M steps across 5 seeds.

Method	Final Return (mean $\pm$ std)	Final Gain	AUC (mean $\pm$ std)
FlashSAC	12,099.38 $\pm$ 676.81	–	9,624.94 $\pm$ 380.14
STAM-FlashSAC ($K = 16$)	**12,600.16** $\pm$ **650.03**	**+4.1%**	**9,887.14** $\pm$ **465.98**

The improvement in this Direct environment is smaller than the gain observed in the main Shadow Hand OpenAI-FF environment. One possible explanation is that the Direct variant provides richer state information to the policy, making the feed-forward FlashSAC baseline more competitive and reducing the marginal benefit of explicit temporal memory. In contrast, the OpenAI-FF setting is more dependent on temporal context, so STAM's ability to aggregate recent observations and actions provides a larger advantage. Overall, the Shadow Hand Direct results support a more nuanced conclusion: STAM appears more beneficial when the observation is less temporally complete, while still offering modest final-performance gains and improved training efficiency in richer-observation settings.

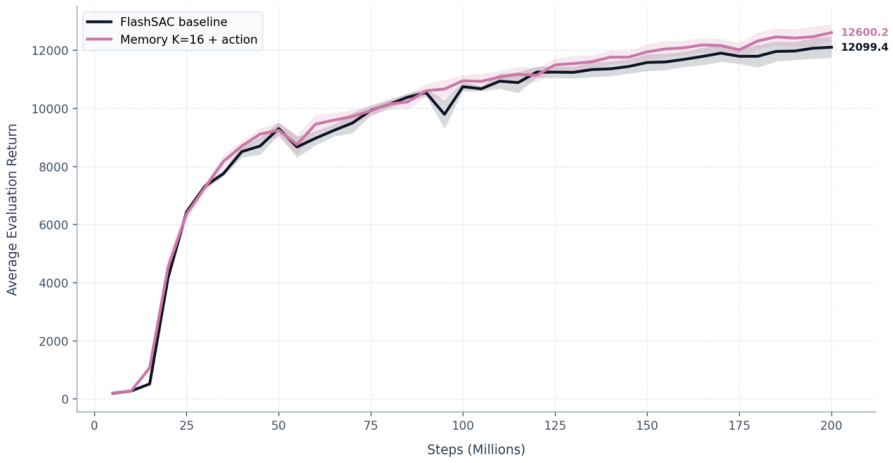


Figure 4: Learning curves on Shadow Hand Direct over 200M environment steps. The FlashSAC baseline is compared with STAM-FlashSAC. Solid lines show average evaluation return across seeds, and shaded regions indicate variability across seeds. STAM-FlashSAC achieves a slightly higher final return and better overall training efficiency, as reflected by its higher area under the learning curve.

B Cross-Embodiment Results on the Allegro Hand

On `Isaac-Repose-Cube-Allegro-Direct-v0`, we compare FlashSAC with STAM-FlashSAC ($K = 16$) with 5 seeds and 200M environment steps each. Table 3 summarises the final evaluation return and AUC for both methods, and Figure 5 shows the corresponding learning curves.

Table 3: Final evaluation return and AUC on Allegro Hand Direct (200M steps, 5 seeds).

Method	Final Return (mean $\pm$ std)	AUC (mean $\pm$ std)
FlashSAC	$5{,}023 \pm 247$	$4{,}215 \pm 259$
STAM-FlashSAC ($K = 16$)	$4{,}617 \pm 51$	$4{,}025 \pm 96$

Unlike in the primary Shadow Hand OpenAI-FF environment, STAM does not improve final return on Allegro Direct ($\Delta = -8.1\%$). One possible explanation is that Allegro Direct provides sufficiently informative instantaneous observations for the feed-forward baseline, reducing the marginal value of explicit short-term memory. Notably, STAM exhibits substantially lower variance across seeds ($\sigma = 51$ vs. 247), indicating more consistent behaviour. We treat this result as a boundary condition rather than a failure: it suggests that STAM's benefit may depend on how much task-relevant temporal information is missing from a single observation.

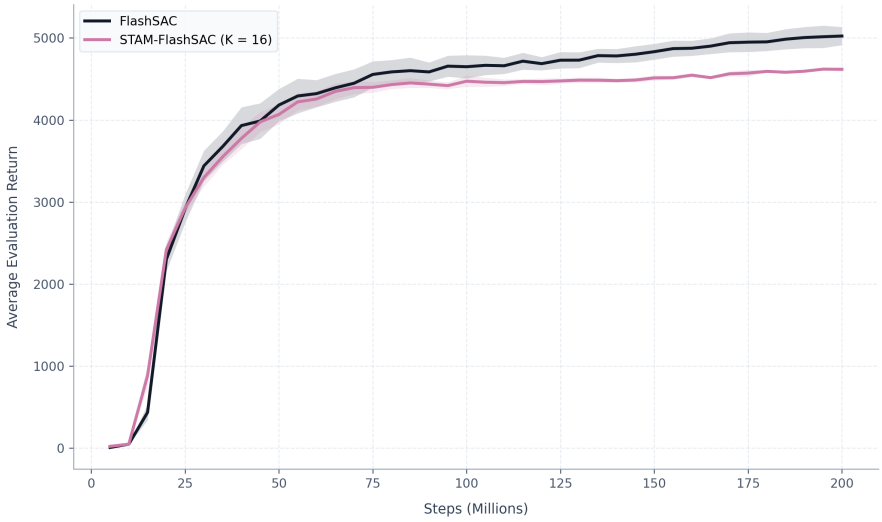


Figure 5: Learning curves on Allegro Hand Direct for STAM-FlashSAC against the FlashSAC baseline. Solid lines show the average evaluation return across seeds; shaded regions indicate variability across seeds.

C Hyperparameters and Implementation Details

Table 4: Default training configuration used for FlashSAC and STAM-FlashSAC unless otherwise stated.

Parameter	Value
Environment	`Isaac-Repose-Cube-Shadow-OpenAI-FF-Direct-v0`
Training steps	50M
Number of parallel environments	1024
Replay buffer size	10^6
Batch size	2048
Warm-up / exploration steps	100k
Updates per interaction step	2
Discount factor γ	0.99
Target update rate τ	0.01
Learning rate	3×10^{-4} with cosine decay
n-step horizon	3
Evaluation frequency	5M steps
Evaluation episodes	1024
STAM context length	$K \in \{4, 8, 16, 32\}$
STAM hidden dimension	128 (ablated: 64, 128, 256)
Hardware	single RTX 4090 GPU

D Ten-Seed Statistical Analysis on the Main Environment

To make the main FlashSAC versus STAM-FlashSAC comparison more transparent, we additionally evaluate the FlashSAC baseline and STAM-FlashSAC ($K = 16$) with 10 paired random seeds on Shadow Hand OpenAI-FF. Table 5 reports the final evaluation return for every seed. STAM-FlashSAC improves over FlashSAC on all 10 paired seeds, with a mean paired improvement of 812.43 final-return points (35.54% relative improvement).

Table 5: Per-seed final evaluation returns for the 10-seed main-environment comparison.

Seed	FlashSAC	STAM-FlashSAC ($K = 16$)	Difference	Relative improvement
0	2,335.90	2,480.11	144.21	6.17%
1	2,248.33	3,158.13	909.81	40.47%
2	2,131.55	3,361.46	1,229.91	57.70%
3	2,169.02	3,058.21	889.20	41.00%
4	2,562.96	3,289.44	726.48	28.35%
5	2,276.28	3,360.09	1,083.81	47.61%
6	2,379.88	2,686.12	306.24	12.87%
7	2,508.19	3,641.83	1,133.64	45.20%
8	2,330.60	3,153.22	822.61	35.30%
9	2,157.97	3,036.38	878.41	40.71%
Mean	2,310.07	3,122.50	812.43	35.54%
Std.	144.92	337.96	346.58	15.77%

Table 6 reports aggregate uncertainty estimates for the same paired comparison, including standard errors and 95% confidence intervals for the method means, paired difference, and relative improvement. Table 7 then reports the paired bootstrap intervals and exact sign-flip test. The 95% confidence interval for the paired final-return difference is $[564.50, 1060.36]$ using the t interval and $[594.61, 999.49]$ using bootstrap resampling. The exact two-sided sign-flip test gives $p = 0.0020$. We report these statistics to avoid over-reliance on mean-only comparisons and to make the seed-level evidence explicit.

Table 6: Aggregate statistics and paired uncertainty estimates for the 10-seed comparison.

Metric	Mean	Std.	SEM	95% CI low	95% CI high
FlashSAC	2,310.07	144.92	45.83	2,206.40	2,413.74
STAM-FlashSAC ($K = 16$)	3,122.50	337.96	106.87	2,880.73	3,364.26
Paired difference	812.43	346.58	109.60	564.50	1,060.36
Relative improvement	35.54%	15.77%	4.99%	24.25%	46.82%

Table 7: Paired bootstrap and sign-flip statistics for the 10-seed main-environment comparison.

Paired statistic	Value	95% interval
Seeds improved	10 / 10	–
Bootstrap paired difference	812.43	[594.61, 999.49]
Bootstrap relative improvement	35.54%	[25.77%, 44.20%]
Exact sign-flip two-sided p-value	0.0020	–

E Computational Cost

We measure wall-clock training time at a fixed budget of 20M environment steps on identical hardware (Table 8). The feed-forward FlashSAC baseline takes 34 minutes, while STAM-FlashSAC takes 43–48 minutes depending on the context length. This corresponds to a slowdown of $1.26\times$–$1.41\times$ ($+26\%$ to $+41\%$). For the best-performing configuration ($K = 16$), the additional cost is accompanied by the main final-return improvement reported in Section 4.2. These timings are operational measurements from our training runs rather than controlled hardware benchmarks.

Table 8: Wall-clock training time at 20M environment steps on identical hardware.

Method	**Time (min)**	**Relative**
FlashSAC	34	$1.00\times$
STAM-FlashSAC ($K = 4$)	43	$1.26\times$
STAM-FlashSAC ($K = 8$)	43	$1.26\times$
STAM-FlashSAC ($K = 16$)	48	$1.41\times$

Table 9 reports the parameter counts and wall-clock time for the frame-stacking variants compared in Section 4.4. STAM has fewer parameters than both frame-stacking baselines, with 2.75M parameters compared with 2.87M for FrameStack using observations only and 3.08M for FrameStack using observations plus previous actions. This suggests that STAM uses its parameters more effectively by learning a recurrent temporal representation instead of directly expanding the input dimension with a fixed history window. However, STAM introduces additional recurrent computation and has a higher wall-clock time than frame stacking. The wall-clock entries in Table 9 are operational measurements from the same training setup and should be interpreted mainly as approximate cost indicators.

F Shuffled Action-History Control

We add a shuffled-action control to test whether the action-history gain comes only from increased input dimensionality or optimization effects. This variant uses the same $K = 16$ context length and action-token dimensionality as aligned action-history STAM, but shuffles the previous-action tokens to break temporal alignment.

Table 10 shows that correctly aligned action history achieves the highest final return, while shuffled actions still improve over observation-only STAM. This suggests that input dimensionality or optimization effects may contribute to the action-history gain, but temporal alignment provides additional benefit.

Table 9: Model size and wall-clock cost for each variant.

Method	Parameter count	Wall-clock time (hr)
FlashSAC	2,467,262	0.478
FrameStack ($K = 16$, obs + action)	3,080,962	0.616
FrameStack ($K = 16$, obs only)	2,874,242	0.579
STAM-FlashSAC ($K = 16$)	2,746,430	0.831

Table 10: Shuffled action-history control on the primary Shadow Hand OpenAI-FF environment. All variants use STAM with $K = 16$ and three seeds.

Configuration	Context dim	Action mode	Final return
STAM-FlashSAC ($K = 16$, obs only)	42	–	2746.0 ± 325.9
STAM-FlashSAC ($K = 16$, shuffled action)	62	shuffled	2847.7 ± 50.5
STAM-FlashSAC ($K = 16$)	62	aligned	2942.6 ± 276.1

G Hidden-Dimension Ablation

We also test the hidden dimension of the STAM memory branch while keeping the context length fixed at $K = 16$. Figure 6 compares hidden dimensions $\{64, 128, 256\}$ over three seeds. The 128-dim setting gives the strongest mean final return in this ablation, while 64 remains close and 256 does not provide an additional gain. We therefore use hidden dimension 128 as the default setting in the main experiments.

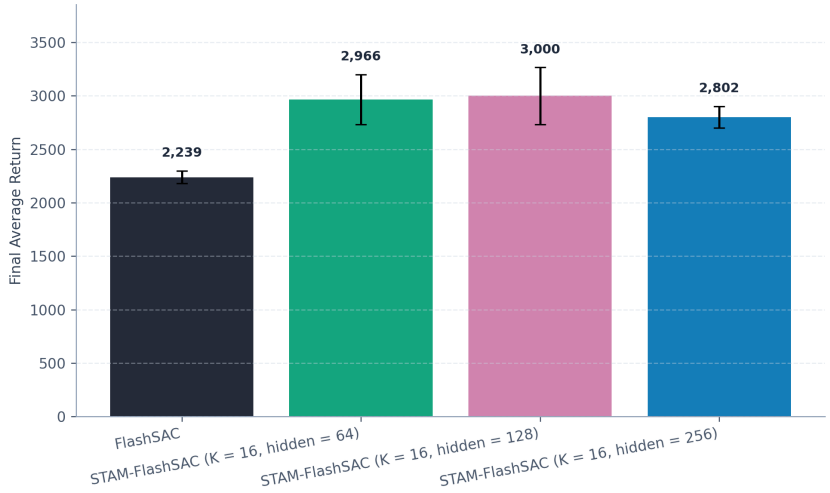


Figure 6: Hidden-dimension ablation for STAM-FlashSAC on the primary Shadow Hand OpenAI-FF environment across 3 seeds.

H Temporal Encoder Ablation

We compare the default single-layer LSTM with GRU and flattened-history MLP encoders under the same $K = 16$ action-observation memory and feature-wise gating setup.

Figure 7 shows that all STAM variants improve over FlashSAC. GRU achieves the highest mean final return (3305.35), followed by MLP (3061.02) and LSTM (2999.90). These results suggest that STAM's benefit is not tied to a specific recurrent cell.

Table 11 reports model size and operational runtime for the same variants. GRU uses fewer trainable actor–critic parameters than LSTM and performs well in this ablation. Because this was a 3-seed follow-up experiment added after the main study, we keep LSTM as the conservative default STAM

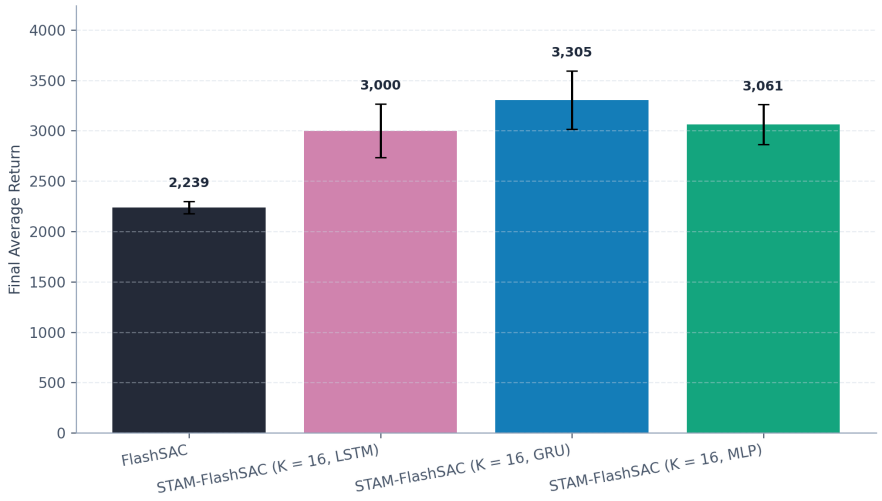


Figure 7: Temporal encoder ablation on the primary Shadow Hand OpenAI-FF environment. Error bars show standard deviation across three seeds.

encoder and present GRU as a promising alternative. The MLP encoder has more parameters than LSTM but only a small mean-return gain, suggesting that explicit recurrence is useful for this short-term memory module.

Table 11: Parameter count and approximate runtime for temporal encoder variants. Runtime is estimated from completed process logs in a shared multi-process environment and should be treated as operational rather than controlled benchmarking.

Encoder	Actor params	Critic params	Actor+critic params	Runtime mean
LSTM	390,268	2,356,162	2,746,430	4.77 h
GRU	365,692	2,331,586	2,697,278	5.13 h
MLP	435,580	2,401,474	2,837,054	4.19 h

I Actor-Only and Critic-Only STAM

To examine where the memory branch should be applied, we compare full STAM against actor-only and critic-only variants on the primary Shadow Hand OpenAI-FF environment. All STAM variants use $K = 16$ with previous actions and are trained for 50M steps with three seeds.

Figure 8 shows that applying STAM to both actor and critic performs best, reaching a final return of 3370. Critic-only STAM is the stronger single-sided variant, reaching 2987, while actor-only STAM reaches 2611. This suggests that temporal context is especially useful for value estimation, but critic-only STAM still remains below the full actor–critic configuration. Overall, the result supports using STAM in both the actor and critic, where memory can jointly improve policy conditioning and value estimation.

J Control Ablation on the N-Step Target

We evaluate the effect of the shared n-step target used by both FlashSAC and STAM-FlashSAC. FlashSAC and STAM-FlashSAC are trained on the Shadow Hand OpenAI-FF task with $n \in \{1, 3\}$, using five seeds and 50M environment steps. All other settings follow Appendix C. For a fair 2×2 comparison, Table 12 reports a matched 5-seed subset for both settings.

Results show that moving from $n = 1$ to $n = 3$ improves both FlashSAC and STAM-FlashSAC. Under the same $n = 3$ setting, STAM-FlashSAC still provides a substantial improvement over FlashSAC, indicating that the benefit of STAM is additive to the shared FlashSAC training configuration.

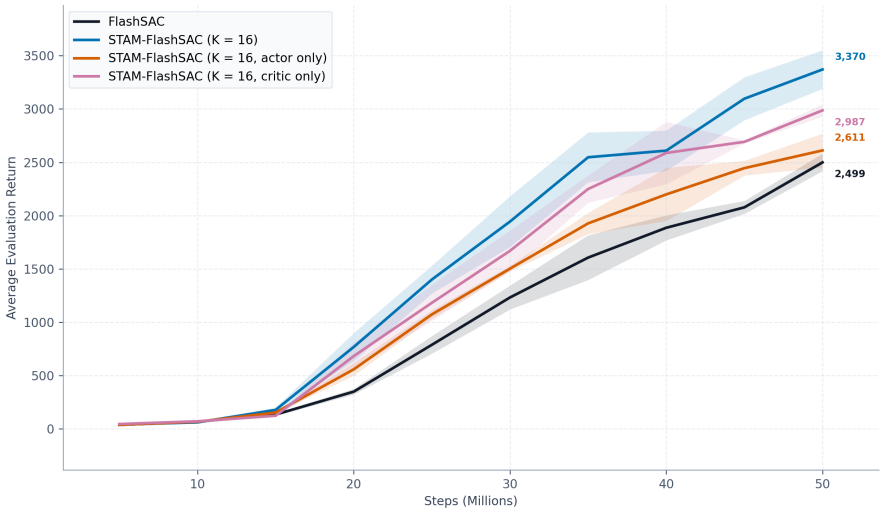


Figure 8: Actor-only and critic-only STAM placement ablation on the primary Shadow Hand OpenAI-FF environment.

Table 12: Final evaluation return (mean $\pm$ std, 5 seeds) at 50M steps.

	$n = 1$	$n = 3$
FlashSAC	255.09 ± 41.58	$2{,}256.38 \pm 142.99$
STAM-FlashSAC ($K = 16$)	289.22 ± 51.40	$3{,}069.47 \pm 349.64$

K Feature-wise Gating vs. Concatenation

We compare the default sigmoid feature-wise gate with a concat fusion variant, which appends the LSTM hidden state directly to the actor and critic feature vectors before the output heads. Both variants use $K = 16$ with action history, and all other settings follow Appendix C.

Figure 9 shows that concat fusion is worse than feature-wise gating across training. In the 3-seed comparison, gate fusion reaches a mean final return of $3{,}370.43$, while concat fusion reaches $2{,}356.80$, corresponding to a decrease of $1{,}013.63$ points (-29.13%). The mean AUC also decreases by 380.57. These results suggest that STAM benefits from using temporal memory as a modulation signal for actor and critic features, rather than merely appending the memory representation to the feature vector.

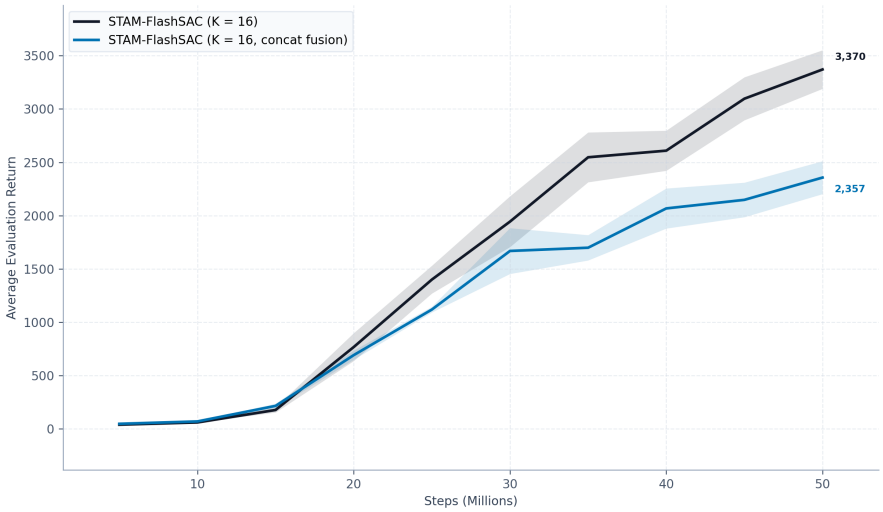


Figure 9: Learning curves for feature-wise gating and concat fusion on the primary Shadow Hand OpenAI-FF environment.

L Limitations

Our study has several limitations. First, the strongest final-return result is shown on one primary environment, the Shadow Hand OpenAI-FF cube reorientation task. We provide a 10-seed paired analysis for the main FlashSAC versus STAM-FlashSAC comparison in Appendix D, but several design ablations are still evaluated with three seeds. Broader claims would require more seeds and additional manipulation tasks. Second, the Shadow Direct and Allegro experiments are intended as boundary-condition analyses: STAM provides modest gains on Shadow Direct, but does not improve final return or AUC on Allegro Direct. These results do not establish universal final-return improvements across environments or embodiments. Third, STAM is evaluated only with state-based observations and short context windows. We do not evaluate visual observations because image encoders would introduce separate perception and representation-learning confounds, substantially increase compute cost, and may require different fusion choices with the STAM memory branch. Extending STAM to visual policies and broader manipulation suites remains an important direction for future work.